

Relational-Core Graph Analytics

<https://arxiv.org/pdf/2609.01525>

1. Problem

- 문제: 그래프 분석을 위해 별도의 Graph DB에 데이터를 복제해야 하는 기존 접근의 비용과 성능 한계
- 기업 데이터는 이미 RDB/Lakehouse의 테이블·FK 형태로 관계를 가지고 있는데, 이를 다시 node/edge 그래프로 변환하는 것이 불필요한 오버헤드
- 왜 중요한가: GraphRAG/AI Agent의 multi-hop query가 늘면서, 데이터를 복제하지 않고 기존 데이터 위에서 그래프 분석을 수행할 필요성이 커짐

2. Method

- 핵심 아이디어: Cypher → SQL로 변환해 기존 relational schema 위에서 그래프 쿼리를 직접 실행
- 제안 시스템:
 - ClickGraph: Cypher → ClickHouse SQL
 - DeltaGraph: Cypher → Databricks/Spark SQL
- 기존 방식과 차이:
 - 기존: Relational DB → ETL → Graph Store → Graph Query
 - 제안: Relational DB → Cypher → SQL → 바로 실행
- zero-ETL, zero-copy 방식으로 기존 SQL 엔진의 columnar storage, vectorized execution, query optimizer를 그대로 활용
- FK, junction table 등 relational schema의 관계를 graph edge로 해석

3. Experiments & Results

- LDBC Social Network Benchmark에서 41개 쿼리 중 26개 실행 가능
- 데이터가 SF1 → SF10으로 10배 증가했을 때 지원 쿼리의 median latency는 약 2.5배 증가
- Interactive Short: 3.6–133 ms, median 23 ms
- Interactive Complex: 44–236 ms, median 130 ms
- Business Intelligence: 41 ms–1.8 s, median 367 ms
- 동일 PostgreSQL 환경에서 native FK-join SQL이 Apache AGE의 node/edge 방식보다 2.4–6.8배 빠름
- 전체 그래프 2-hop query: 0.56 s vs. 2.21 s, 약 3.9배 차이
- 정확성 테스트:
 - openCypher TCK read scenario 402개 통과
 - LDBC backend parity check 22개 통과
- 한계 성능:
 - variable-length recursive query는 데이터 증가에 따라 급격히 느려짐
 - BI3: 1.8 s → 241 s, 약 130배 증가

4. Insight

- 핵심 기여: Graph DB를 별도로 만들지 않고 기존 relational data를 graph로 해석한 뒤 Cypher를 SQL로 변환해 대규모 graph analytics를 수행
- 한계:
 - 현재 read-only OLAP 중심
 - 41개 LDBC 쿼리 중 26개만 지원
 - recursive/variable-length path에서 성능 문제가 큼
 - 주로 single-node 환경에서 평가
 - Neo4j와 직접적인 동일 조건 비교는 아님